

31.12.25

Trading (Strategies, Execution, Quotes, QuotesGeneration, Positions, Trades, Orders)

Это целый кластер автономных торговых сервисов:

1. Trading Strategies Service

Отвечает за реализацию стратегий (TradeStrategy, DiscreteRandomTradeStrategy, EventDrivenRandomStrategy, EventDrivenRandomStrategy01 и т.п.).

Зона ответственности:

- **инкапсулирует торговую логику на уровне стратегии (как реагировать на тики, свечи, сделки, ордера);**
- **управляет позицией через Position/IPosition, считает PnL (realized/unrealized);**
- **формирует ордера и сделки (CreateOrder, ExecuteOrder, OnDealExecuted);**
- **публикует доменные события (через EventHub-ориентированные стратегии, например EventHubStrategy, которые вы используете совместно с StrategiesExecutionService), такие как:**
 - **исполненная сделка (TradeExecutedEvent);**
 - **закрытая сделка (DealClosedEvent);**
 - **изменение позиции (PositionChangedEvent);**
 - **создание ордера (OrderCreatedEvent).**

Стратегия — это автономный «микросервис в памяти»: она не знает ни про RazorPages, ни про БД, ни про UI. Она просто принимает входные события (тик/свеча/исполнение), обновляет своё состояние и через события сообщает наружу.

1. Trading Execution Service (StrategiesExecutionService)

Фоновой сервис, который исполняет стратегии.

Зона ответственности:

- **подписывается на QuoteGeneratedEvent через IDynamicSubscriptionManager и EventHub;**
- **по каждой пришедшей котировке находит соответствующий тикер в EventHubTickerManager и передаёт тики во все стратегии по этому тикеру;**
- **запускает и останавливает стратегии (StartTrading, StopTrading) централизованно;**
- **собирает и логирует статистику производительности (сколько тиков обработано, скорость, сколько стратегий и тикеров задействовано).**

Фактически это отдельный сервис-исполнитель, который изолирует вычислительную логику стратегий от генерации котировок, UI и хранения

данных. Он живёт на потоке `BackgroundService` и общается с остальными только через `EventHub`.

1. Quotes Generation Service (QuotesGeneratorService + EventHubTicker / EventHubTickerManager)

Это связка сервисов котировок:

- **EventHubTicker** — «умный тикер», наследник `Ticker`, который:
 - генерирует локальную котировку (`GenerateNextQuote + GetCurrentQuote`);
 - при генерации публикует доменные события:
 - `QuoteGeneratedEvent` (рыночная котировка);
 - `ChartUpdateEvent` (сигнал для перерисовки графиков).
- **QuotesGeneratorService** — фоновый сервис-генератор:
 - циклически обходит все тикеры в `EventHubTickerManager`;
 - инициирует генерацию новых котировок;
 - публикует `QuoteGeneratedEvent` в `EventHub`;
 - логирует ход генерации и итерации.

Таким образом, **QuotesGeneration** — это отдельный сервисный контур: он только создаёт рыночные данные и выбрасывает их в `EventHub`, не зная, кто именно их будет слушать (стратегии, UI, мониторинг и т.д.).

1. Trading Monitoring Service (TradingMonitorService)

Сервис пассивного мониторинга торговых событий.

Зона ответственности:

- подписывается на:
 - `TradeExecutedEvent`,
 - `QuoteGeneratedEvent`,
 - `PositionChangedEvent`;
- ведёт счётчик событий, печатает агрегированную статистику и диагностические сообщения;
- не вмешивается в бизнес-логику, только наблюдает и логирует.

Это отдельный автономный сервис-наблюдатель, подключённый к `EventHub` как «прослушиватель шины».

1. Console Quotes Service (QuotesConsoleService)

Ещё один автономный сервис-подписчик, фокусированный на котировках.

Зона ответственности:

- подписка (через `IDynamicSubscriptionManager`) на `quote:generated`;

- красивая раскрашенная консольная визуализация потоков котировок с простейшими метриками (изменение цены, %-изменение, цвет по тикеру, счётчик и скорость);
- итоговые статистики при остановке.

Это полностью изолированный «консольный UI» для стрима котировок, который знает только про `EventHub` и тип `QuoteGeneratedEvent`.

1. Trading Factory Services (`GS.Trade.Core.TradingFactory` и `TradingPlatform.Services.TradingFactory`)

Две реализации фабрики торговых сущностей:

- ядро (`GS.Trade.Core.TradingFactory`) — создает `Trade`, `Deal`, `Tick`, `CandleStick`, `Order` из `core`-моделей;
- платформа (`TradingPlatform.Services.TradingFactory`) — создает `SimpleTrade`, `SimpleDeal`, `SimpleTick`, `SimpleCandle`, `SimpleOrder`.

Зона ответственности:

- инкапсулируют создание доменных объектов, отделяя стратегию и остальной код от конкретных реализаций классов;
- позволяют подменять реализации (модели ядра vs упрощённые модели в платформе) без переписывания логики.

Итого по блоку `Trading`:

У вас не «монолитный трейдинг-модуль», а набор автономных сервисов:

- сервисы стратегий;
- сервис исполнения стратегий;
- сервис генерации котировок;
- консольный мониторинг котировок;
- сервис мониторинга трейдинга;
- фабрики торговых сущностей.

Все они не вызывают друг друга напрямую, а общаются через `EventHub` посредством доменных событий.

Charts (Visualization, Rendering, Ajax-Request, `ChartDirector`)

Это блок сервисов, отвечающих за визуализацию цен и динамики:

1. Chart Container / Chart Windows (`TradingPlatform.Charts.ChartContainer` и отдельные окна графиков)

Зона ответственности:

- хранит коллекцию «окон графиков» для разных тикеров/таймфреймов;
- каждое окно умеет отрисовывать себя в RazorChartViewer (ChartDirector) и опционально принять новые котировки;
- отделяет логику построения графика от RazorPage и от источника данных.

1. RazorPage Finance (Charts UI + Ajax)

Страница /Finance и FinanceModel — это «тонкий» UI-слой над сервисом графиков:

- при OnGet:
 - создаёт массив RazorChartViewer по количеству окон в ChartContainer;
 - вызывает _chartContainer[i].Render(viewer), не зная деталей построения графика;
- обработчик OnGetChartHtml:
 - при каждом запросе (через fetch) создаёт свежий RazorChartViewer для каждого окна;
 - вызывает Render (или потенциально AddNextQuote + Render);
 - возвращает чистый HTML-фрагмент.

1. Ajax-клиент на странице /Finance

Зона ответственности:

- периодически (каждые ~1.4 сек) дергает Finance?handler=ChartHtml;
- подменяет innerHTML контейнера диаграмм;
- тем самым организует лёгкий live-update графиков без перезагрузки страницы.

Таким образом, Charts-подсистема — это отдельный визуализационный сервис: она не торгует, не хранит данные, а просто, по запросу UI, обращается к ChartContainer и строит графики. Сигналы о необходимости обновления приходят через события (ChartUpdateEvent), а данные могут подтягиваться из InMemoryDatabase или других источников.

RazorPage (Visualization, Rendering, Trades, Orders, все торговые сущности, полный UI, Charts Rendering)

Это целый UI-сервис на Razor Pages, который живёт поверх Event-ориентированного ядра:

1. Trades UI Service (Trades RazorPage + API /api/trading/trades)

Зона ответственности:

- визуализирует последние трейды (VisualTrade) с фильтрами по Ticker/Strategy;
- умеет:
 - пагинацию;
 - ручное обновление;
 - автообновление через fetch (/api/trading/trades?...), анимации таблицы, подсветку «NEW» сделок;
- не хранит сделки сам: использует InMemoryTradingDatabase как автономный бэкенд-сервис.

1. Orders UI Service (Orders RazorPage)

Зона ответственности:

- таблица ордеров (VisualOrder) с фильтрами и пагинацией;
- визуальная дифференциация по статусу (Filled, Pending, Cancelled, Rejected);
- также читает только из InMemoryTradingDatabase.

1. Deals UI Service (Deals RazorPage)

Зона ответственности:

- список закрытых сделок (VisualDeal) с фильтрами и пагинацией;
- подсветка PnL (profit/loss);
- получает данные только из InMemoryTradingDatabase.

1. Positions UI Service (Positions RazorPage)

Зона ответственности:

- отображает сводную информацию по позициям (на основе PositionViewModel, который строится из TickerStrategySummary);
- считает и показывает агрегаты:
 - Total PnL (Open/Closed/Total);
 - Total Exposure;
 - число открытых позиций;
- фильтры по тикеру/стратегии, галочка OnlyOpenPositions, сортировки и пагинация;
- UI логика: расчёт Exposure, PnL%, состояния OPEN/CLOSED — всё это делает PageModel, но входные данные идут из автономной InMemoryDatabase.

1. Summary UI Service (Summary RazorPage)

Зона ответственности:

- высокоуровневая статистика по тикер+стратегия: TickerStrategySummary;
- фильтры, пагинация, подсветка PnL и количества сделок;
- также опирается на InMemoryTradingDatabase.

1. Layout / Navigation

Общий Razor-layout, который «сшивает» все UI-сервисы:

- навигация между Finance, Summary, Trades, Orders, Deals, Positions;
- но сам layout не знает ни о стратегиях, ни о событиях — только маршруты страниц.

Итого RazorPage-блок — это законченный UI-микросервис:

он не генерирует котировки и не исполняет стратегии, а только «читает» подготовленные данные из InMemoryDatabase и строит визуализации (таблицы, графики) плюс даёт удобный Аjax-интерфейс.

InMemoryDataBase — самостоятельная разработка

InMemoryTradingDatabase — это отдельный, полностью автономный сервис хранения и агрегации данных в памяти.

Зона ответственности:

- **Подписка на доменные события через EventHub / DynamicSubscriptionManager:**
 - TradeExecutedEvent;
 - DealClosedEvent;
 - OrderCreatedEvent;
 - PositionChangedEvent;
 - QuoteGeneratedEvent.
- **Внутренние структуры:**
 - **ConcurrentDictionary для трейдов, сделок, ордеров и суммариев:**
 - _trades: [ticker:strategy] -> List<VisualTrade>;
 - _deals: [ticker:strategy] -> List<VisualDeal>;
 - _orders: [ticker:strategy] -> List<VisualOrder>;
 - _summaries: [ticker:strategy] -> TickerStrategySummary;
 - _tickerPrices для текущих цен по тикерам.
- **Логика обработки событий:**
 - ProcessTradeEvent — создаёт VisualTrade, добавляет его в коллекции, пересчитывает summary (позиция, openTrades, totalTrades), дергает TradeAdded и SummaryUpdated;
 - ProcessDealEvent — создаёт VisualDeal, аккумулирует ClosedPnL в соответствующем summary, поднимает DealAdded и SummaryUpdated;
 - ProcessOrderEvent — создаёт VisualOrder, добавляет, дергает OrderAdded;
 - ProcessPositionEvent — обновляет позицию и openPnL по тикеру+стратегии, вызывает SummaryUpdated;
 - ProcessQuoteEvent — обновляет текущую цену в суммариях и пересчитывает OpenPnL для открытых позиций.

- **Публичный API для UI/других сервисов:**
 - **GetSummaries()** — отдаёт перечисление **TickerStrategySummary**;
 - **GetTrades(...), GetDeals(...), GetOrders(...)** — отдаёт **PagedResult<T>** для визуализации в **RazorPage**;
 - события для **SignalR/реалтайма**: **SummaryUpdated, TradeAdded, DealAdded, OrderAdded**.

Фактически это самостоятельная in-memory «БД-шина»:

она не знает ни о RazorPages, ни о стратегиях внутри — только о событиях EventHub и своих структурах.

UI-слой видит её просто как InMemoryTradingDatabase и не знает, что под капотом всё работает через EventHub и фоновые подписки.

Total Logging

Логирование не прицеплено к одному месту — это распределённый «сквозной» сервис, интегрированный во все автономные компоненты:

- **EventHub (InProcessEventHub) логирует:**
 - публикацию сообщений;
 - количество найденных хэндлеров;
 - ошибки в обработчиках без падения всего процесса.
- **DynamicSubscriptionManager, CompositeKeyRouter, EventHandlerRegistry** — тоже логируют регистрацию/подписки, ошибки резолвинга хэндлеров.
- **Trading-сервисы (StrategiesExecutionService, QuotesGeneratorService, QuotesConsoleService, TradingMonitorService, InMemoryTradingDatabase)** ведут детальный журнал:
 - запуск/остановка сервисов;
 - подписки на ключи;
 - статистика обработки (количество котировок, событий, тиков, скорость);
 - предупреждения о проблемных ситуациях (не найден тикер, нет стратегии, пустая стратегия и т.п.).

Важный момент: логирование интегрировано на уровне каждого сервиса, но не ломает их автономность. Они используют общий логгер-инфраструктуру (ILogger, Serilog), но всё равно остаются независимыми и общаются друг с другом через EventHub, а не через прямые вызовы.

EventHub — центральное архитектурное звено

Всё перечисленное выше работает не через прямые ссылки служб друг на друга, а через EventHub и его окружение:

- **InProcessEventHub + Channel** — асинхронная in-process шина сообщений;
- **Key-routing:**
 - статическая маршрутизация через атрибуты [Handles] и EventHandlerRegistry;
 - динамическая маршрутизация через DynamicSubscriptionManager и CompositeKeyRouter;
- интерфейсы IMessage, IMessageHandler<>, IKeyRouter, IEventHub, IDynamicSubscriptionManager, IEventHandlerRegistry формируют ядро шины;
- Keys и MessageKeyAttribute/IStaticKeyMessage — стандартизуют ключи сообщений.

Через EventHub связаны:

- генерация котировок;
- исполнение стратегий;
- торговые события (trades, deals, orders, positions);
- in-memory хранилище;
- визуализации (через QuoteGeneratedEvent/ChartUpdateEvent);
- мониторинг и логирование на уровне доменных событий.

EventHub не знает ничего о торговых объектах, стратегиях, RazorPages или InMemoryDatabase — он просто маршрутизирует сообщения по ключу к подписчикам. Благодаря этому каждая область остаётся сервисом со своей отдельной ответственностью.

Во всём приложении прослеживается единая линия: всё ориентировано на автономные сервисы (Trading, Execution, Quotes, InMemoryDatabase, RazorPage-UI, Monitoring), а коммуникации между ними идут через EventHub.

- Стратегии не знают про UI.
- UI не знает про внутренности стратегий.
- InMemoryDatabase не знает про способ генерации котировок.
- Генератор котировок не знает, кто их будет использовать.

Все они «договорились» только о формате сообщений (Events) и ключах EventHub.

Это и есть настоящая событийно-ориентированная архитектура с чёткими сервисными границами и минимальными связями между ними.

1. Архитектурное описание для документации

1.1. Общая идея

Система построена по событийно-ориентированной архитектуре вокруг общего механизма передачи сообщений (EventHub).

Каждый доменный сервис (котировки, стратегии, исполнение, хранилище, UI, мониторинг) автономен и взаимодействует с остальными только через события и их ключи.

EventHub не содержит доменной логики и не знает о типах торговых сущностей (ордеры, сделки, стратегии и т.п.). Его задача — маршрутизация сообщений по ключам к соответствующим подписчикам.

1.2. Базовые интерфейсы шины сообщений

Ядро шины формируют следующие интерфейсы:

- **IMessage** — базовый контракт любого сообщения.
Описывает минимальный набор метаданных (например, идентификатор, время возникновения и т.п.).
- **IMessageHandler<TMessage>** — обработчик сообщений.
Любой компонент, который хочет реагировать на событие, реализует этот интерфейс для соответствующего типа сообщения.
- **IKeyRouter** — маршрутизатор по ключам.
Отвечает за сопоставление «ключ сообщения → зарегистрированные обработчики».
- **IEventHub** — центр подписки и публикации событий.
Предоставляет методы Publish(...) и Subscribe(...), а внутри опирается на IKeyRouter и реестр обработчиков.
- **IDynamicSubscriptionManager** — управление подписками во время работы.
Позволяет компонентам динамически подписываться или отписываться от нужных событий.
- **IEventHandlerRegistry** — реестр обработчиков.
Содержит информацию о том, какие обработчики к каким сообщениям/ключам привязаны.

1.3. Стандартизация ключей сообщений

Для того чтобы разные сервисы могли согласованно общаться, используется единый механизм ключей сообщений:

- **Keys** — набор констант/значений, описывающих ключи сообщений (например, "Quotes.Generated", "Trading.OrderPlaced", "UI.Chart.Update" и т.д.).
- **MessageKeyAttribute / IStaticKeyMessage** — механизм явного связывания типа сообщения и его ключа. Возможные подходы:
 - **сообщение реализует интерфейс IStaticKeyMessage и возвращает свой ключ;**
 - **ключ задаётся через атрибут MessageKeyAttribute на типе сообщения.**

Благодаря этому:

- **EventHub не должен «знать» про каждый тип события;**
- **новые типы событий можно добавлять, не меняя код шины;**
- **ключи становятся единым «языком» между сервисами.**

1.4. Основные доменные сервисы и их взаимодействие

1.4.1. Сервис котировок (Quotes)

Ответственность:

- **генерация потоков котировок (реальные/симулированные);**
- **публикация событий о новых ценах.**

Типичные события:

- **QuoteGeneratedEvent (ключ, например, Keys.Quotes.Generated).**

Роль в системе:

- **источник данных для стратегий;**
- **источник обновлений для визуализации (через соответствующие события).**

1.4.2. Сервис стратегий (Trading / Strategies)

Ответственность:

- **приём рыночных данных (например, котировок);**
- **выполнение логики торговых стратегий;**
- **генерация команд/событий для исполнения, логирования и обновления состояния.**

Типичные входные события:

- **QuoteGeneratedEvent (новые цены);**
- **события об исполнении ордеров, изменении позиций и т.п.**

Типичные выходные события:

- **заявки на исполнение (например, OrderRequestedEvent);**
- **события о состоянии стратегии (например, StrategyStateChangedEvent);**
- **сервисные события для мониторинга (например, StrategyErrorEvent).**

Роль в системе:

- **ядро прикладной логики;**

- концентратор, который реагирует на поток входящих событий и порождает новые.

1.4.3. Сервис исполнения (Execution)

Ответственность:

- приём команд на выставление/изменение/отмену ордеров;
- взаимодействие со сторонними сервисами (биржа, симулятор и т.д.);
- публикация событий по результатам исполнения.

Типичные входные события:

- OrderRequestedEvent (из стратегий);
- события управления (приостановка, возобновление, отмена и т.д.).

Типичные выходные события:

- OrderPlacedEvent;
- OrderExecutedEvent;
- TradeCreatedEvent и т.д.

Роль в системе:

- «мост» между стратегиями и фактическим движением ордеров/сделок;
- источник правдивой информации о состоянии заявок.

1.4.4. InMemory хранилище (InMemoryDatabase / State Store)

Ответственность:

- хранение актуального состояния:
 - активные ордера;
 - позиции;
 - история сделок;
 - состояния стратегий;
 - последние котировки (для UI, быстрой аналитики и т.д.).

Типичные входные события:

- OrderPlacedEvent, OrderExecutedEvent, TradeCreatedEvent;
- PositionChangedEvent;
- QuoteGeneratedEvent (для кэша цен);
- доменные события стратегий (состояние, ошибки, параметры).

Роль в системе:

- единое in-memory зеркало текущего доменного состояния;
- источник данных для UI и мониторинга (через запросы к этому хранилищу или через вторичные события).

1.4.5. UI / Визуализации (RazorPages, Charts)

Ответственность:

- отображение данных пользователю:
 - котировки и графики;
 - текущие позиции и ордера;
 - история сделок;
 - состояние стратегий;
 - системный мониторинг (ошибки, задержки, метрики).

Типичные входные события:

- QuoteGeneratedEvent / ChartUpdateEvent — для обновления графиков;
- события об изменении позиций, ордеров, сделок;
- события мониторинга и ошибок.

Роль в системе:

- тонкий клиент поверх событийной модели;
- не содержит бизнес-логики, а только подписывается на нужные потоки событий.

1.4.6. Мониторинг и логирование (Monitoring / Logging)

Ответственность:

- приём доменных и технических событий;
- агрегация, алертинг, журналирование.

Типичные входные события:

- события стратегий (успех/ошибка, изменение состояния);
- события исполнения (ошибки, задержки, отмены);
- системные события (health-check, метрики).

Роль в системе:

- сквозное наблюдение за состоянием всех остальных сервисов;
- минимальное влияние на доменную логику (подписчик, а не интеграционный «центр»).

1.5. Схема «кто какие события публикует и слушает» (словесный вариант блок-схемы)

Если представить блок-схему в текстовом виде, получится примерно так:

1. Сервис котировок:

- **публикует:**
 - **QuoteGeneratedEvent**
- **подписывается:**
 - **опционально — на служебные события (запуск/остановка генерации).**

1. Сервис стратегий:

- **подписывается:**
 - **QuoteGeneratedEvent (новые цены);**
 - **события об исполнении (OrderExecutedEvent, TradeCreatedEvent, PositionChangedEvent);**
- **публикует:**
 - **OrderRequestedEvent (команды в Execution);**
 - **StrategyStateChangedEvent, StrategyErrorEvent (в Monitoring, UI, InMemoryStore).**

1. Сервис исполнения:

- **подписывается:**
 - **OrderRequestedEvent (от стратегий);**
- **публикует:**
 - **OrderPlacedEvent, OrderExecutedEvent, TradeCreatedEvent, OrderCancelledEvent и т.п.**

1. InMemory хранилище:

- **подписывается:**
 - **все ключевые доменные события (ордера, сделки, позиции, котировки, состояние стратегий);**
- **публикует:**
 - **опциональные агрегирующие события (например, PortfolioUpdatedEvent, SnapshotCreatedEvent).**

1. UI:

- **подписывается:**
 - **события для визуализации: котировки, графики, позиции, ордера, сделки, состояние стратегий;**
 - **события мониторинга и ошибок;**

- публикует:
 - команды пользователя (опционально, если UI инициирует действия), например `UserCommandEvent` → дальше уже обрабатывается соответствующим доменным сервисом.

1. Мониторинг:

- подписывается:
 - широкий спектр событий со всех сервисов (ошибки, изменение состояния, метрики);
- публикует:
 - специализированные события для алертинга/отчётов (например, `AlertRaisedEvent`).

1.6. Архитектурные принципы

- Сервисы общаются только через `EventHub`.
 - Шина сообщений (`IEventHub` + ключи + реестр обработчиков) не зависит от доменной модели.
 - Новые доменные области добавляются через:
 - определение своих типов событий;
 - привязку этих типов к ключам;
 - регистрацию обработчиков через общий механизм подписки.
 - Каждый сервис может развиваться и тестироваться изолированно, подменяя `EventHub` на тестовый/локальный.
-

2.

Текст для презентации (структура и содержание слайдов)

Ниже — примерная структура; можно взять как есть и просто раскидать по слайдам.

1. Название и цель

Тема: «Событийно-ориентированная архитектура торговой системы»

Содержание (тезисы):

- Архитектура вокруг единого `EventHub`
- Сервисы автономны, общаются событиями
- Цель: минимальные связи, максимальная расширяемость

2. Роль `EventHub`

Тезисы:

- **EventHub** — центр маршрутизации сообщений
- **Не содержит бизнес-логики**
- **Работает только с интерфейсами:**
 - **IMessage**
 - **IMessageHandler<>**
 - **IKeyRouter**
 - **IEventHub**
 - **IDynamicSubscriptionManager**
 - **IEventHandlerRegistry**

3. Ключи сообщений

Тезисы:

- **Единый язык взаимодействия — ключи событий (Keys)**
- **Связка типа и ключа:**
 - **MessageKeyAttribute**
 - **IStaticKeyMessage**
- **Позволяет добавлять новые события без изменения шины**

4. Картина сервисов

Тезисы:

- **Сервисы:**
 - **Котировки (Quotes)**
 - **Стратегии (Trading)**
 - **Исполнение (Execution)**
 - **InMemoryDatabase**
 - **UI (RazorPages, графики)**
 - **Мониторинг**
- **Все соединены только через EventHub**

5. Сервис котировок

Тезисы:

- **Генерирует рыночные данные**
- **Публикует: QuoteGeneratedEvent**
- **Не знает, кто будет эти котировки использовать:**
 - **стратегии**
 - **UI**
 - **InMemory-хранилище**

6. Сервис стратегий

Тезисы:

- **Подписывается:**
 - **на котировки**

- на события исполнения и позиций
- Публикует:
 - заявки на исполнение (**OrderRequestedEvent**)
 - события о состоянии стратегий (**StrategyStateChangedEvent**, ошибки)
- Не знает о UI и хранилище

7. Сервис исполнения

Тезисы:

- Принимает команды от стратегий
- Работает с внешними исполнителями (биржа/симулятор)
- Публикует:
 - **OrderPlacedEvent**
 - **OrderExecutedEvent**
 - **TradeCreatedEvent**
- Источник правдивых данных об ордерах

8. InMemoryDatabase

Тезисы:

- Подписан на все ключевые доменные события
- Держит:
 - ордера, позиции, сделки
 - состояние стратегий
 - последние котировки
- Служит быстрым источником актуальных данных для UI и аналитики

9. UI и визуализация

Тезисы:

- Подписывается на:
 - котировки и обновления графиков
 - позиции, ордера, сделки
 - состояние стратегий и ошибки
- Не содержит доменной логики
- Может инициировать действия через события команд

10. Мониторинг и логирование

Тезисы:

- Подписан на события со всех сервисов
- Агрегирует:
 - ошибки
 - метрики
 - состояние стратегий и исполнения

- **Обеспечивает наблюдаемость без вмешательства в доменную логику**

11. Преимущества архитектуры

Тезисы:

- **Слабая связанность сервисов**
- **Простое добавление новых компонентов и событий**
- **Тестируемость: каждый сервис можно запускать отдельно**
- **EventHub переиспользуем как общую коммуникационную шину**

12. Заключение

Тезисы:

- **Вся система — набор сервисов, связанных только событиями**
- **EventHub и ключи сообщений — устойчивый договор между ними**
- **Архитектура упрощает развитие, масштабирование и поддержку**